Université Savoie Mont Blanc,
UFR SceM
INFO702, M1 INFO

October 27, 2026

Warm-up C++, POO

(Passage par référence, classes, surcharge, polymorphisme dynamique)

Question 1. Utilisation des références

Réécrivez le code suivant en utilisant au maximum des références. Réécrivez ensuite le code en C++ objet, avec constructeur et operator+=.

```
struct Point {
  double x;
  double y;
};

struct Vecteur {
  double vx;
  double vy;
};


int main()
{
  Point p0;
  Vecteur v0 = { 1.0, 0.0 };
  cree( &p0, 4.0, 1.0 );
  translate( &p0, v0 );
}
```

```
void cree( Point* ptr_p, double x0,
    double y0 )
{
  ptr_p->x = x0;
  ptr_p->y = y0;
}
void translate( Point* ptr_p, Vecteur v )
{
  ptr_p->x += v.vx;
  ptr_p->y += v.vy;
}
```

Question 2. Classes ; surcharge d'opérateurs

Nous avons vu en cours comment faire une classe `Vecteur`, équipée de différents opérateurs. Nous allons faire une classe compagnon `Matrice`, qui représente une matrice carrée 2×2.

$$M := \begin{pmatrix} m_{00} & m_{01} \\ m_{10} & m_{11} \end{pmatrix}.$$

On veut pouvoir faire les opérations suivantes avec nos `Matrices`.

- *attributs*: Choisissez les attributs/données membres nécessaires
- *construction*: Initialiser une matrice à zéro, ou avec 4 coefficients, ou à l'aide de deux vecteurs colonnes.
- *accesseurs*: Accès en lecture/écriture aux coefficients de la matrice. On précisera en paramètres la ligne (0 ou 1) et la colonne (0 ou 1). On surchargera l'opérateur `operator()` en lecture et écriture.
- *opérations arithmétiques de matrices*: on peut multiplier une matrice par un double, additionner deux matrices, multiplier deux matrices. On rappelle que si A et B sont deux matrices de taille $n \times n$, et $P = A * B$, alors le coefficient en ligne i et colonne j de P vaut:

$$p_{ij} = \sum_{k=0\ldots n-1} a_{ik} * b_{kj} \quad \text{(ici } n = 2\text{, donc)} \quad = a_{i0} * b_{0j} + a_{i1} * b_{1j}$$

- *opérations matrices vecteur*: on peut multiplier une matrice par un `Vecteur`, ce qui retourne un `Vecteur`

- *matrices de rotation*: les matrices de rotation sont des matrices particulières (leur déterminant vaut 1) dont une propriété notable est qu'elles peuvent tourner les vecteurs dans le plan. On les forme ainsi, si θ est l'angle de rotation voulu:

$$R_\theta := \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}.$$

  Ecrire une méthode *statique* `rotation( double theta )` qui retourne la matrice R_θ.

  On peut maintenant tourner un `Vecteur` de $\pi/4$ radians (45) ainsi:

```
Vecteur v( 3.0, 1.0 );
Matrice R45 = Matrice::rotation( std::acos(-1.0) / 4.0 );
Vecteur w = R45 * v;
```

- *déterminant, inverse*: le déterminant d'une matrice carré est important dans de nombreuses applications en calcul scientifique. Ecrire une méthode `det()` qui le calcule. Ecrire ensuite une méthode `inverse()`. Si le déterminant calculé est nul, la méthode lèvera `std::domain_error`. Cette vérification exacte ne détecte pas une matrice mal conditionnée.

Question 3. Polymorphisme dynamique : jeu de dames

On voudrait modéliser un jeu de dames, composé d'un plateau de $10x10$ cases, alternant noir et blanc, de pions de deux couleurs, et éventuellement de dames (2 pions l'un sur l'autre).

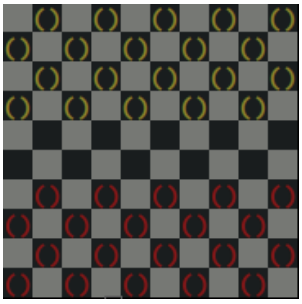
Chaque `Case` est donc noire ou blanche. Au maximum, elle a une `Piece` (abstraite) dessus (concrètement un `Pion` ou une `Dame`).

```
class Case {
  bool m_color; //< noir (false) ou blanc
  // Pointeur vers la piece, ou nullptr si la case est vide
  Piece* m_piece;
public:
  Case( bool color = false );
  void change( Piece* p );
  void affiche() const;
};
struct Damier {
  Case cases[10][10];
  Damier(); // initialise les cases
  void initJoueur( int j ); // et le joueur
  void affiche() const;
};
```

```
struct Piece {
  virtual ~Piece() = default;
  virtual void affiche() const = 0;
};
class Pion : public Piece {
  int m_joueur;
public:
  Pion( int joueur ); // Joueur 0 ou 1
  virtual void affiche() const override;
};
struct Dame : public Piece {
  Dame( int joueur ); // Joueur 0 ou 1
  virtual void affiche() const override;
};
```

On rappelle la fonction `console` qui permet de faire des affichages en couleur dans le terminal (`https://stackoverflow.com/questions/2616906/how-do-i-output-coloured-text-to-a-linux-terminal`)

```
string console( string code )
{ return "\033[" + code + "m"; }
auto   normal      = console("0"); // revient à normal
auto   clignotant  = console("5"); // clignotant
auto   texte_rouge = console("1;31"); // rouge gras
auto   texte_jaune = console("1;33"); // jaune gras
auto   fond_noir   = console("40");   // fond noir
auto   fond_gris   = console("47");   // fond gris
```



La classe `Piece` est définie avant `Case`.

1. Complétez le code ci-dessus (les corps des méthodes) pour que l'on puisse créer un `Damier` et le visualiser ainsi. Un pion sera représenté par '()' gras, une dame par '@@' gras et clignotant.

```
...
Damier D;
D.initJoueur( 0 );
D.initJoueur( 1 );
D.affiche();
```

Dans un premier temps, initialisez les cases et créez les pions avec `new Pion(joueur)` ; `change` se contente de mémoriser le pointeur reçu. On ne libère pas encore les pièces dans cette première version.

2. Quel est le problème lorsque le damier est détruit ? Que se passe-t-il si `change` remplace une pièce ou si `initJoueur` est appelée deux fois ? Un pointeur brut indique-t-il à lui seul qui possède l'objet ?

3. Modifiez la classe `Case` pour qu'elle possède sa pièce avec un `std::unique_ptr<Piece>`. Adaptez `change` et la création des pions. Expliquez la destruction lors d'un remplacement et à la fin de la vie du damier, le rôle du destructeur virtuel et les conséquences sur la copie.

4. Une autre solution consiste à conserver les pièces dans le damier et à laisser les cases les désigner par des pointeurs non propriétaires. Qui doit alors gérer la durée de vie des pièces ? Que faut-il faire des pointeurs des cases lorsqu'une pièce est retirée ?

Question 4. Durée de vie et propriété d'une ressource

On veut représenter une suite de mesures de taille fixée à la construction.

1. Écrire une classe `Mesures` qui possède un tableau dynamique de `double`, initialisé à zéro. Fournir `size()` et deux surcharges de `operator[]` pour un accès mutable ou constant. L'indice doit être strictement inférieur à la taille.

2. Réaliser une première version avec `new[]` et `delete[]`. Pourquoi la copie par défaut est-elle incorrecte ? Interdire explicitement copie et déplacement pour cette version. Que se passe-t-il à la sortie du bloc ?

3. Réécrire la classe avec `std::vector<double>`. Quel est alors l'effet d'une copie ? Modifier une copie et vérifier que l'original est inchangé. Pourquoi cette version n'a-t-elle pas besoin d'un destructeur explicite ?

Question 5. Polymorphisme dynamique : formes et détection de collision

On va regarder un algorithme "probabiliste" pour savoir si deux formes s'intersectent. L'idée est de ne tester que quelques points aléatoirement sur une des formes. Pour chacun, on regarde s'il appartient à l'autre forme. Si oui, on est sûr que les formes s'intersectent (on a détecté une *collision*), sinon on n'est sûr de rien (!) mais plus on tirera de points aléatoirement, plus on sera "sûr". Ce genre d'algorithme est utilisé dans les cas suivants:

- les formes que l'on compare ne sont pas assez simples pour permettre un test analytique facile,
- les formes que l'on compare sont variées (on ne peut pas tester précisément tous les cas possibles d'intersection),
- on ne dispose pas forcément d'un budget de calcul illimité, mais seulement un certain temps (genre dans un jeu temps-réel)
- ce n'est pas "grave" si on se trompe (genre dans un jeu temps-réel)

Un résultat positif prouve l'intersection ; un résultat négatif ne prouve pas que les formes sont disjointes. Même avec beaucoup de tirages, une intersection de probabilité presque nulle peut ne jamais être détectée.

Un inconvénient (il y en a toujours) est qu'il faut que chaque forme fournisse quelques services:

- une méthode qui dise si oui ou non un `Point` est à l'intérieur de cette forme
- une méthode qui retourne un nouveau `Point` aléatoire dans la forme. Bien sûr, il vaut mieux que le tirage aléatoire soit le plus uniforme possible pour que l'algorithme maximise ses chances de détecter la collision.
- (optionnel) une méthode qui retourne la boîte englobante de la forme.

1. Proposer une classe abstraite `Shape` qui impose les deux opérations précédentes à toute sous-classe.

2. Ecrire la forme concrète `Rectangle`, qui définit un rectangle aligné avec les axes entre un point bas p et un point haut q.

   On utilisera la fonction ci-dessous qui retourne un nombre pseudo-aléatoire discret dans l'intervalle $[0, 1]$.

   ```
   double rand01() {
     return static_cast<double>(std::rand()) / RAND_MAX; // <cstdlib>
   }
   ```

3. On pourrait écrire aussi une forme concrète `Disk` sans trop de difficulté, là encore en la centrant en un point c, et en donnant un rayon r.

4. Ecrire maintenant la fonction `checkCollisionOnePoint( const Shape & f1, const Shape & f2 )` qui teste si un point de `f1` appartient à `f2`. Ecrire ensuite la fonction `checkCollisionManyPoints( const Shape & f1, const Shape & f2, int n )` qui teste alternativement si `n` points de `f1` appartiennent à `f2` et inversement.

   Pourquoi alterner entre les tests ? Quelle est la probabilité que notre algorithme fonctionne ?

5. On peut ensuite enrichir considérablement nos formes en introduisant des transformations du plan, des groupages, des intersections, ce qui permet de construire des formes très complexes.

   Ecrire une classe `TranslatedShape`, qui est une `Shape`, et qui référence une autre `Shape`, et qui fait la translation d'une forme par un vecteur donné à la construction (u, v).

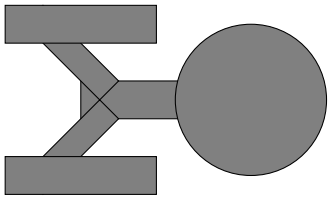
   Ecrire une classe `RotatedShape`, qui est une `Shape`, et qui référence une autre `Shape`, et qui fait la rotation d'une forme par un angle donné à la construction r.

   Ecrire une classe `UnionShape`, qui est une `Shape`, et qui référence deux autres formes, et qui représentre l'union des deux formes.

   Les formes référencées doivent rester vivantes tant que les objets qui les utilisent existent. Le tirage dans l'union choisit une des deux formes avec probabilité approximativement $1/2$: est-il uniforme sur l'union ? Pourquoi le tirage par rejet dans une intersection quelconque peut-il ne jamais terminer ?

6. On voit qu'on a maintenant le moyen de créer des formes très complexes en combinant ces objets très simples. L'algorithme de détection de collision n'a finalement pas changé du tout !!

   Créer la situation suivante.

7. Les tirages des deux formes sont supposés indépendants. Si la probabilité qu'un tirage de la première appartienne à la seconde vaut p_1, et si la probabilité inverse vaut p_2, calculer la probabilité de détecter une collision après n tirages dans chaque forme. Que se passe-t-il pour deux disques tangents ?